

# Практика RAG

# Практика RAG

## Вступление

- Задача блока: научиться строить **полный RAG-пайплайн** на примере кулинарных книг.
- Что у нас есть:
  - PDF/сканы кулинарных книг;
  - OpenAI GPT модели для понимания текста и изображений;
  - Embeddings + FAISS для поиска;
  - Ноутбук RAG with OpenAI.ipynb как базовый код.
- Что в итоге: ассистент, который отвечает на вопросы по рецептам, техникам, заменам ингредиентов и т.д.

# Зачем нужен RAG и почему не хватает GPT

Чистый GPT:

- не знает наши конкретные книги;
- «галлюцинирует» рецепты;
- не может «листануть» PDF локально.

**RAG (Retrieval-Augmented Generation):**

- сначала находит релевантные фрагменты в наших данных;
- потом строит ответ, опираясь именно на них.

Для нас: RAG = способ «подключить» кулинарные книги к модели.

# Кейс: ассистент по кулинарным книгам

Датасет: одна или несколько кулинарных книг в PDF/сканах.

Типичные вопросы:

- «Подбери ужин из курицы и риса до 40 минут».
- «Чем заменить сливочное масло, если его нет?»
- «Объясни этот рецепт простыми словами».

Цель системы:

- честные, полезные ответы;
- ссылки на конкретные рецепты/страницы.

# Общая архитектура RAG-системы

## **1.Data layer:**

1. загрузка PDF → конвертация в изображения;
2. чтение страниц с GPT (vision) → текст + структура.

## **2.Processing layer:**

1. извлечение рецептов;
2. фильтрация мусора (оглавления, реклама);
3. нормализация: ингредиенты, шаги, время, теги.

## **3.Retrieval layer:**

1. генерация эмбеддингов;
2. FAISS-индекс + метаданные.

## **4.Generation layer:**

1. обработка вопроса;
2. поиск в индексе;
3. комбинирование контента;
4. генерация ответа GPT;
5. форматирование.

## А. Офлайн-пайплайн: готовим базу рецептов и индекс

## В. Онлайн-пайплайн: от вопроса к ответу

PDF / сканы  
книг

Изображения  
страниц

GPT-Vision  
OCR + понимание

Структурированные  
рецепты

Фильтрация /  
нормализация

DocStore  
(чистая база)

Embeddings  
рецептов

FAISS-индекс +  
Metadata Store

Вопрос  
пользователя

Эмбединг  
запроса

FAISS:  
top-k рецептов

Robust retrieval /  
логирование

Комбинированный  
контекст

Промпт для  
GPT (system+user)

Ответ  
GPT

Вывод ответа  
пользователю

# А. Офлайн-пайплайн: готовим базу рецептов и индекс

## Шаги офлайн-пайплайна:

1. Кулинарные книги (PDF/сканы)
2. Конвертация PDF → изображения
3. GPT-Vision: чтение страниц (OCR + понимание)
4. Структурированные рецепты (JSON/таблица)
5. Фильтрация и нормализация рецептов
6. Генерация эмбеддингов для рецептов
7. Построение FAISS-индекса
8. Хранилище метаданных (DocStore + Metadata Store)

PDF / сканы  
книг

Изображения  
страниц

GPT-Vision  
OCR + понимание

Структурированные  
рецепты

Фильтрация /  
нормализация

DocStore  
(чистая база)

Embeddings  
рецептов

FAISS-индекс +  
Metadata Store

# Шаг 1. PDF / сканы книг

## Что есть:

- 1+ кулинарных книг в виде:
  - PDF (возможно, со сканированными страницами),
  - отдельных изображений.

**Цель:** привести эти книги к формату, с которым GPT и embedding'ы могут работать.

Источник данных для всей RAG-системы — одна или несколько кулинарных книг в формате PDF или отсканированных страниц.

Файлы размещаются, например, в Google Drive и подключаются к среде выполнения (Colab).

На этом шаге мы не меняем содержимое книг — только обеспечиваем доступ к ним из кода.

Ключевой результат шага — корректный путь к файлу PDF (`pdf\_path`), который дальше используется при конвертации в изображения.

Д.Ю. Хартьян. ТГУ

PDF / сканы  
книг

Изображения  
страниц

GPT-Vision  
OCR + понимание

Структурированные  
рецепты

Фильтрация /  
нормализация

DocStore  
(чистая база)

Embeddings  
рецептов

FAISS-индекс +  
Metadata Store



# Шаг 1. PDF / сканы книг — пример кода

```
# Mount the drive
from google.colab import drive
drive.mount('/content/drive')
# Change directory to this folder
%cd /content/drive/MyDrive/GenAI/RAG/RAG with OpenAI
# Define the path to the PDF and the output folder for images
pdf_path = "Things mother used to make.pdf"
output_folder = "images"

# Convert the PDF into images and store the image paths
image_paths = pdf_to_images(pdf_path, output_folder)
```

# Шаг 2. Изображения страниц

**Вход:** PDF книги.

**Выход:** набор изображений по страницам:

•page\_001.jpg,page\_002.jpg...

**Почему так:**

- GPT vision-модель удобнее кормить **картинками**, чем пытаться парсить PDF напрямую.
- На картинке есть и текст, и форматирование, и иллюстрации блюда.

Мы превращаем каждую страницу PDF-книги в отдельное изображение (обычно JPEG).

Для этого используем библиотеку `pdf2image` и утилиту `poppler`, которые умеют корректно рендерить страницы PDF.

Функция `pdf\_to\_images` проходит по всем страницам и сохраняет их как `page1.jpg`, `page2.jpg`, ... в заданную папку.

Результат шага — список путей к изображениям (`image\_paths`), с которыми далее будет работать GPT-Vision.

PDF / сканы  
книг

Изображения  
страниц

GPT-Vision  
OCR + понимание

Структурированные  
рецепты

Фильтрация /  
нормализация

DocStore  
(чистая база)

Embeddings  
рецептов

FAISS-индекс +  
Metadata Store

## Шаг 2. Изображения страниц — пример кода

```
# Install the pdf2image library for converting PDF files to images
!pip install pdf2image
# Install the poppler-utils package, required by pdf2image to work with
PDF files
!apt-get install -y poppler-utils
# Import the libraries
from pdf2image import convert_from_path
import os
# Function to converts pdfs into images and stores the image paths
def pdf_to_images(pdf_path, output_folder):
    # Create the output folder if it doesn't exist
    if not os.path.exists(output_folder):
        os.makedirs(output_folder)

    # Convert PDF into images
    images = convert_from_path(pdf_path) # Convert each page of the PDF to
an image
    image_paths = []

    # Save images and store their paths
    for i, image in enumerate(images):
```

# Шаг 3. GPT-Vision — OCR + понимание

**Вход:** одна картинка страницы.

**Выход:** структурированный объект «рецепт» или сигнал «рецепта нет».

Здесь важен **промт** к GPT:

- Объясняем, что на картинке — страница кулинарной книги.
- Просим: если есть рецепт — извлечь:
  - title — название;
  - ingredients — список ингредиентов;
  - steps — пошаговая инструкция;
  - cook\_time, servings, возможно category и т.д.
- Просим вернуть в **строгом формате** (например, JSON) или чётко размеченный текст.

Мы кодируем изображение в base64 и передаём его в `chat.completions` как `image\_url` вместе с системным промптом.

Системный промпт задаёт задачу: извлечь из картинки информацию о рецепте (название, ингредиенты, шаги и т.д.).

Результат — текстовый ответ модели, содержащий описание или структуру рецепта для одной страницы.

PDF / сканы  
книг

Изображения  
страниц

GPT-Vision  
OCR + понимание

Структурированные  
рецепты

Фильтрация /  
нормализация

DocStore  
(чистая база)

Embeddings  
рецептов

FAISS-индекс +  
Metadata Store

# Шаг 3. GPT-Vision — OCR + понимание — пример кода

```
# Install the openAI library
!pip install openai
# Import the libraries
from openai import OpenAI
import base64
# Set up connection to OpenAI API
client = OpenAI(
    api_key=api_key, # Use the provided API key for authentication
)
# Specify the model to be used
model = "gpt-4o-mini"
# Read and encode one image
image_path = "images/page23.jpg" # Path to the image to be encoded

# Encode the image in base64 and decode to string
with open(image_path, "rb") as image_file:
    image_data = base64.b64encode(image_file.read()).decode('utf-8')
image_data
# Define the system prompt
system_prompt = """
Please analyze the content of this image and extract any related recipe
```

# Шаг 4. Структурированные рецепты

**Задача:** автоматизировать шаг А3 на весь массив страниц.

**Вход:** список `image_paths`.

**Выход:** список структурированных объектов «страница → рецепты/текст».

В ноутбуке:

- цикл/функция, которая:
  - для каждой картинки вызывает GPT-vision,
  - собирает результат в массив/таблицу.

На этом шаге получается **сырой набор данных:** рецепты + случайные страницы (оглавление, реклама и т.п.).

Запускаем обработку для всех страниц: для каждой картинки вызываем GPT и сохраняем результат в список ``extracted_recipes``.

PDF / сканы  
книг

Изображения  
страниц

GPT-Vision  
OCR + понимание

Структурированные  
рецепты

Фильтрация /  
нормализация

DocStore  
(чистая база)

Embeddings  
рецептов

FAISS-индекс +  
Metadata Store

# Шаг 4. Структурированные рецепты — пример кода

```
# Define improved system prompt
system_prompt2 = """
Please analyze the content of this image and extract any related recipe
information into structure components.
Specifically, extra the recipe title, list of ingredients, step by step
instructions, cuisine type, dish type, any relevant tags or metadata.
The output must be formatted in a way suited for embedding in a Retrieval
Augmented Generation (RAG) system.
"""

# Call the api to extract the information
response = client.chat.completions.create(
    model = model,
    messages = [
        # Provide the system prompt
        {"role": "system", "content": system_prompt2},

        # The user message contains both the text and image URL / path
        {"role": "user", "content": [
            "This is the image from the recipe page",
            {"type": "image_url",
             "image_url": {"url": f"data:image/jpeg;base64,{image_data}"},
             "url": f"data:image/jpeg;base64,{image_data}"},
            "Д.Ю. Хартьян. ТГУ"}
        ]
    ]
)
```

# Шаг 5. Фильтрация / нормализация

**Цель:** оставить только то, что реально является рецептами / полезными сущностями.

**Что делаем:**

- выбрасываем страницы:
  - без ингредиентов и шагов;
  - слишком короткие или неструктурированные (эвристики по длине, ключевым словам);
- проверяем «полноту» рецепта:
  - есть ли название;
  - есть ли хотя бы несколько ингредиентов и шагов.

**Выход: DocStore** – «чистое хранилище рецептов

Мы фильтруем полученные ответы по ключевым словам (например, `ingredients`, `instructions`, `recipe title`).

Только записи, где присутствуют признаки рецепта, попадают в список `filtered\_recipes`. Затем сохраняем очищенный список в JSON (`recipe\_info.json`) — это основа дальнейшей нормализации и анализа.

PDF / сканы  
книг

Изображения  
страниц

GPT-Vision  
OCR + понимание

Структурированные  
рецепты

Фильтрация /  
нормализация

DocStore  
(чистая база)

Embeddings  
рецептов

FAISS-индекс +  
Metadata Store



# Шаг 5. Фильтрация / нормализация — пример кода

```
# Filter out non-recipe content based on key recipe-related terms.  
Удаление страниц без рецептов.  
filtered_recipes = []  
  
for recipe in extracted_recipes:  
    # Check if the extracted content contains any key recipe-related terms  
    if any(keyword in recipe["recipe_info"].lower() for keyword in  
["ingredients",  
  
"instructions",  
  
"recipe title"]):  
        # If it does, add it to the filtered list  
        filtered_recipes.append(recipe)  
  
    # Print a message for non-recipe content  
    else:  
        print(f"Skipping recipe: {recipe['image_path']}")  
# import json library  
import json  
# Define the output file path
```

# Шаг 6. DocStore (чистая база)

DocStore — это чистое хранилище рецептов, удобное для дальнейшего поиска и генерации. На этом шаге мы работаем уже с отфильтрованными данными, загружая их из `recipe\_info.json`.

Каждый элемент DocStore содержит текст рецепта и привязку к исходному изображению/странице.

Такое хранилище можно переиспользовать без повторного вызова GPT-Vision: оно становится устойчивой базой знаний.

PDF / сканы  
книг

Изображения  
страниц

GPT-Vision  
OCR + понимание

Структурированные  
рецепты

Фильтрация /  
нормализация

DocStore  
(чистая база)

Embeddings  
рецептов

FAISS-индекс +  
Metadata Store

# Шаг 6. DocStore (чистая база) — пример кода

```
# import json library
import json
# Define the output file path
output_file = "recipe_info.json"

# Write the filtered list to a json file
with open(output_file, "w") as json_file:
    json.dump(filtered_recipes, json_file, indent = 4)
# Load the filtered recipes
with open("recipe_info.json", "r") as json_file:
    filtered_recipes = json.load(json_file)
```

# Шаг 7. Embeddings рецептов

**Вход:** DocStore — список рецептов.

**Выход:** массив векторов (эмбеддингов) + соответствие id → вектор.

Мы берём `recipe\_info` каждого рецепта и отправляем список текстов в модель `text-embedding-3-large`.

Результат — матрица эмбеддингов (`embedding\_matrix`), где каждая строка соответствует одному рецепту.

PDF / сканы  
книг

Изображения  
страниц

GPT-Vision  
OCR + понимание

Структурированные  
рецепты

Фильтрация /  
нормализация

DocStore  
(чистая база)

Embeddings  
рецептов

FAISS-индекс +  
Metadata Store

# Шаг 7. Embeddings рецептов — пример кода

```
# import libraries
import numpy as np
# Load the filtered recipes
with open("recipe_info.json", "r") as json_file:
    filtered_recipes = json.load(json_file)
# Generate embeddings for each recipe
recipe_texts = [recipe["recipe_info"] for recipe in filtered_recipes] # Extract
the text content of each recipe

# Call the API to generate embeddings for the recipe texts
embedding_response = client.embeddings.create(
    input = recipe_texts, # Provide the list of recipe texts as input
    model = "text-embedding-3-large" # Specify the embedding model to use
)
# Extract the embeddings
embeddings = [data.embedding for data in embedding_response.data]
embeddings
# Convert the embeddings to numpy array
embedding_matrix = np.array(embeddings)
embedding_matrix
```

# Шаг 8. FAISS-индекс + Metadata Store

**Вход:** список эмбеддингов + список recipe\_id.

**Выход:**

**1.FAISS-индекс** (векторная структура для быстрых запросов).

**2.Metadata Store** (словарь / таблица: id → рецепт + метаданные).

Архитектурно:

- FAISS** отвечает только за «найти похожие вектора».
- Вся **бизнес-информация** (название блюда, страница, текст) живёт отдельно.

Мы инициализируем индекс `IndexFlatL2` подходящей размерности и добавляем в него матрицу эмбеддингов.

Индекс сохраняется на диск, а параллельно мы формируем `metadata` — отдельное хранилище с текстом рецепта и путём к изображению.

Такое разделение (FAISS для векторов, JSON для метаданных) облегчает масштабирование и перенастройку retrieval-слоя.

PDF / сканы  
книг

Изображения  
страниц

GPT-Vision  
OCR + понимание

Структурированные  
рецепты

Фильтрация /  
нормализация

DocStore  
(чистая база)

Embeddings  
рецептов

FAISS-индекс +  
Metadata Store

# Шаг 8. FAISS-индекс + Metadata Store — пример кода

```
# Install the faiss-cpu library
!pip install faiss-cpu
# Import the faiss library
import faiss
# Print the embedding matrix shape
print(f"Embedding matrix shape: {embedding_matrix.shape}")
# Initialize the FAISS index for similarity search
index = faiss.IndexFlatL2(embedding_matrix.shape[1]) # Create a FAISS
index with L2 distance metric
index.add(embedding_matrix) # Add the embeddings to the index
# Save the FAISS index to a file
faiss.write_index(index, "filtered_recipe_index.index")
# Save the metadata for each recipe
metadata = [{'recipe_info': recipe['recipe_info'], # Include recipe
information
            'image_path': recipe['image_path']} for recipe in
filtered_recipes] # Include image path

# Write metadata to a JSON file with indentation
with open("recipe_metadata.json", "w") as json_file:
    json.dump(metadata, json_file, indent = 4)
```

# Итоги части А: офлайн-пайплайна

Что мы сделали в офлайн-пайплайне:

- Превратили кулинарные книги (PDF/сканы) в набор изображений страниц.
- Считали страницы через GPT-Vision и извлекли из них структурированные рецепты.
- Отфильтровали нерелевантные страницы и собрали DocStore — чистую базу рецептов.
- Закодировали рецепты в эмбединги и построили FAISS-индекс векторов.
- Разнесли числовой индекс (FAISS) и бизнес-метаданные (Metadata Store).

Что у нас есть на выходе части А:

- DocStore с рецептами, готовыми для поиска и генерации ответов.
- Embedding-матрица рецептов и сохранённый FAISS-индекс.
- Metadata Store, который связывает каждый вектор с конкретным рецептом и страницей книги.

Ключевая идея: офлайн-пайплайн превращает «сырые» PDF и сканы в структурированную и индексируемую базу знаний, с которой затем работает онлайн-пайплайн RAG.



# В. Онлайн-пайплайн: от вопроса к ответу

## Шаги онлайн-пайплайна:

1. Вопрос пользователя
2. Эмбеddинг запроса
3. Поиск top-k в FAISS
4. Обработка и логирование результатов (robust retrieval)
5. Комбинированный контекст из найденных рецептов
6. Построение промпта для GPT
7. Генерация ответа моделью
8. Возврат ответа пользователю (RAG-функция)

Вопрос  
пользователя

Эмбеddинг  
запроса

FAISS:  
top-k рецептов

Robust retrieval /  
логирование

Комбинированный  
контекст

Промпт для  
GPT (system+user)

Ответ  
GPT

Вывод ответа  
пользователю

# Шаг 1. Вопрос пользователя

Онлайн-пайплайн начинается с текстового запроса пользователя — естественного вопроса на любом языке.

Запрос описывает кулинарную задачу: например, подобрать рецепт, уточнить технику или найти альтернативу ингредиенту.

На уровне кода запрос — это обычная строка (`query` или `query1`, `query2`), которую мы затем передаём в RAG-функции.

Важно отделять интерфейс (где пользователь вводит вопрос) от внутренней логики retrieval и генерации.

Вопрос  
пользователя

Эмбеddинг  
запроса

FAISS:  
top-k рецептов

Robust retrieval /  
логирование

Комбинированный  
контекст

Промпт для  
GPT (system+user)

Ответ  
GPT

Вывод ответа  
пользователю

# Шаг 1. Вопрос пользователя — пример кода

```
# Test the rag system
query1 = "How to make the best chocolate cake?"
response = rag_system(query1, index, metadata, system_prompt3)
get_gpt_response()
# Test with a different query
query2 = "I want something vegan"
response = rag_system(query2, index, metadata, system_prompt3)
get_gpt_response()
```

## Шаг 2. Эмбеддинг запроса

Чтобы искать по смыслу, запрос пользователя также кодируется в вектор эмбеддинга.

Мы используем ту же модель `text-embedding-3-large`, что и для рецептов, чтобы запрос и рецепты жили в одном векторном пространстве.

На первом этапе демонстрации мы генерируем эмбеддинг для строки `query` и преобразуем его в вектор `query\_vector` нужной формы. Этот вектор затем используется для поиска ближайших рецептов в FAISS-индексе.

Вопрос  
пользователя

Эмбеддинг  
запроса

FAISS:  
top-k рецептов

Robust retrieval /  
логирование

Комбинированный  
контекст

Промпт для  
GPT (system+user)

Ответ  
GPT

Вывод ответа  
пользователю

## Шаг 2. Эмбеddинг запроса — пример кода

```
# Generate the embeddings for the query
query = "How to make bread?"
k = 5 # Number of top results to retrieve
query_embedding = client.embeddings.create(
    input = [query],
    model = "text-embedding-3-large"
).data[0].embedding
print(f"The query embedding is {query_embedding}\n")
query_vector = np.array(query_embedding).reshape(1, -1) # Convert embedding
to a 2D numpy array for FAISS
print(f"The query vector is {query_vector}\n")

# Search the FAISS index for the nearest neighbors
distances, indices = index.search(query_vector, min(k, len(metadata))) #
Perform the search
print(f"The distances are {distances}\n")
print(f"The indices are {indices}\n")

# Store the indices and distances
stored_indices = indices[0].tolist()
stored_distances = distances[0].tolist()
print(f"The stored indices are {stored_indices}\n")
print(f"The stored distances are {stored_distances}\n")
```

## Шаг 3. Поиск top-k в FAISS

После получения эмбединга запроса мы выполняем поиск ближайших векторов рецептов в FAISS-индексе.

Функция ``query_embeddings`` инкапсулирует этот шаг: она ищет ``k`` ближайших соседей с помощью ``index.search``.

Мы также ограничиваем ``k`` длиной массива метаданных, чтобы не запрашивать больше документов, чем существует в хранилище.

На этом шаге мы получаем расстояния и индексы (``distances``, ``indices``), которые определяют, какие рецепты потенциально релевантны запросу.

Вопрос  
пользователя

Эмбединг  
запроса

FAISS:  
top-k рецептов

Robust retrieval /  
логирование

Комбинированный  
контекст

Промпт для  
GPT (system+user)

Ответ  
GPT

Вывод ответа  
пользователю

## Шаг 3. Поиск top-k в FAISS — пример кода

```
# Define a function to query the embeddings
def query_embeddings(query, index, metadata, k = 5):
    # Generate the embeddings for the query
    query_embedding = client.embeddings.create(
        input = [query],
        model = "text-embedding-3-large"
    ).data[0].embedding
    print(f"The query embedding is {query_embedding}\n")
    query_vector = np.array(query_embedding).reshape(1, -1)
    print(f"The query vector is {query_vector}\n")

    # Search faiss index
    distances, indices = index.search(query_vector, min(k, len(metadata)))
    # print(f"The distances are {distances}\n")
    # print(f"The indices are {indices}\n")

# Test the retrieval system
query = "chocolate query"
results = query_embeddings(query, index, metadata)
print(f"The results are {results}")
```

## Шаг 4. Robust retrieval

Чтобы работать с результатами поиска устойчиво, мы сохраняем индексы и расстояния в отдельные списки.

Это позволяет дополнительно анализировать найденные рецепты, логировать результаты и при необходимости реализовывать собственные пороги качества.

Функция `query_embeddings` фильтрует индексы, проверяя, что они находятся в допустимом диапазоне (`0 ≤ i < len(metadata)`). На выходе функция возвращает удобную структуру `results`: список пар `(recipe_info, distance)` для дальнейшей обработки.

Вопрос  
пользователя

Эмбединг  
запроса

FAISS:  
top-k рецептов

Robust retrieval /  
логирование

Комбинированный  
контекст

Промпт для  
GPT (system+user)

Ответ  
GPT

Вывод ответа  
пользователю



## Шаг 4. Robust retrieval — пример кода

```
# Store the indices and distances
stored_indices = indices[0].tolist()
stored_distances = distances[0].tolist()
print(f"The stored indices are {stored_indices}\n")
print(f"The stored distances are {stored_distances}\n")

# # Print the metadata content
# print("The metadata content is")
# for i, dist in zip(stored_indices, stored_distances):
#     if 0 <= i < len(metadata):
#         print(f"Distance: {dist}, Metadata: {metadata[i]['recipe_info']}")

# Return the results
results = [(
    metadata[i]['recipe_info'], dist) for i, dist in zip(
        stored_indices, stored_distances) if 0 <= i < len(metadata)]
return results
```

# Шаг 5. Комбинированный контекст

Модель GPT не работает напрямую с списком документов — ей нужен текстовый контекст. Функция ``combined_retrived_content`` объединяет тексты всех найденных рецептов в один большой блок.

Мы соединяем ``recipe_info`` из каждого результата через двойной перевод строки, чтобы сохранить читаемую структуру.

Такой комбинированный контекст затем используется в промпте как «доказательная база» для ответа модели.

Вопрос  
пользователя

Эмбединг  
запроса

FAISS:  
top-k рецептов

Robust retrieval /  
логирование

Комбинированный  
контекст

Промпт для  
GPT (system+user)

Ответ  
GPT

Вывод ответа  
пользователю

## Шаг 5. Комбинированный контекст — пример кода

```
# Combine the results into a single string
def combined_retrived_content(results):
    combined_content = "\n\n".join([result[0] for result in results]) # Join the re
information with double newlines
    return combined_content

# Get the combined content from results
combined_content = combined_retrived_content(results)
print(f"The combined content is {combined_content}")
```

# Шаг 6. Построение промпта

System-prompt задаёт роль модели: здесь она выступает как опытный шеф-повар, который даёт советы по готовке.

Мы явно указываем, что модель должна опираться только на переданный комбинированный контент (``combined_content``).

Если информации недостаточно, промпт предписывает честно ответить «не знаю», а не придумывать детали.

В конце мы добавляем ссылку на текст запроса (``{query}``), чтобы модель понимала, на какой вопрос нужно ответить.

Вопрос  
пользователя

Эмбеddинг  
запроса

FAISS:  
top-k рецептов

Robust retrieval /  
логирование

Комбинированный  
контекст

Промпт для  
GPT (system+user)

Ответ  
GPT

Вывод ответа  
пользователю

## Шаг 6. Построение промпта — пример кода

```
# Define the system prompt
system_prompt3 = f"""
You are highly experienced and expert chef specialized in providing cooking advice.
Your main task is to provide information precise and accurate on the combined
content.
You answer directly to the query using only information from the provided
{combined_content}.
If you don't know the answer, just say that you don't know.
Your goal is to help the user and answer the {query}
"""
```

# Шаг 7. Генерация ответа GPT

Функция `generate_response` связывает всё вместе на уровне LLM: системный промпт, текст запроса и комбинированный контекст.

Мы вызываем `client.chat.completions.create` с выбранной моделью и тремя сообщениями в диалоге.

Температура `0` делает ответы максимально детерминированными и воспроизводимыми для учебных целей.

На выходе мы получаем объект `response`, из которого затем извлекаем и отображаем текст ответа.

Вопрос  
пользователя

Эмбединг  
запроса

FAISS:  
top-k рецептов

Robust retrieval /  
логирование

Комбинированный  
контекст

Промпт для  
GPT (system+user)

Ответ  
GPT

Вывод ответа  
пользователю

## Шаг 7. Генерация ответа GPT — пример кода

```
# Define function to retrieve a response from the API
def generate_response(query, combined_content, system_prompt):
    response = client.chat.completions.create(
        model = model,
        messages = [
            {"role": "system", "content": system_prompt3}, # Provide system prompt
            {"role": "user", "content": query}, # Provide the query as user input
            {"role": "assistant", "content": combined_content} # Provide the combined
        ],
        temperature = 0, # Set temperature to 0 for deterministic output
    )
    return response

# Get the results from the API
query = "How to make bread?"
combined_content = combined_retrived_content(results)
response = generate_response(query, combined_content, system_prompt3)
# Display the outcome
get_gpt_response()
# Get the results
query = "Get me the best chocolate cake recipe"
combined_content = combined_retrived_content(results)
```

# Шаг 8. RAG-функция и вывод ответа

Функция ``rag_system`` реализует полный контур Retrieval-Augmented Generation в одном вызове. Она последовательно выполняет три шага: retrieval (``query_embeddings``), объединение контента (``combined_retrived_content``) и генерацию (``generate_response``).

Снаружи пользователю достаточно передать текстовый ``query`` и указать индекс, метаданные и системный пром프트 — остальное происходит внутри.

Тестовые вызовы с ``query1`` и ``query2`` демонстрируют, как одна и та же RAG-функция отвечает на разные кулинарные запросы.

Вопрос  
пользователя

Эмбеddинг  
запроса

FAISS:  
top-k рецептов

Robust retrieval /  
логирование

Комбинированный  
контекст

Промпт для  
GPT (system+user)

Ответ  
GPT

Вывод ответа  
пользователю



# Шаг 8. RAG-функция и вывод ответа — пример кода

```
# Build the function for Retrieval-Augmented Generation (RAG)
def rag_system(query, index, metadata, system_prompt, k = 5):
    # Retrieval System: Retrieve relevant results based on the query
    results = query_embeddings(query, index, metadata, k)

    # Content Merge: Combine the retrieved content into a single string
    combined_content = combined_retrived_content(results)

    # Generation: Generate a response based on the query and combined content
    response = generate_response(query, combined_content, system_prompt)

    # Return the generated response
    return response

# Test the rag system
query1 = "How to make the best chocolate cake?"
response = rag_system(query1, index, metadata, system_prompt3)
get_gpt_response()

# Test with a different query
query2 = "I want something vegan"
response = rag_system(query2, index, metadata, system_prompt3)
get_gpt_response()
```

# Итоги части В: онлайн-пайплайна

Что делает онлайн-пайплайн:

- Принимает вопрос пользователя (query) из интерфейса — чата, формы, CLI.
- Преобразует запрос в эмбединг и ищет top-k ближайших рецептов в FAISS-индексе.
- Применяет robust retrieval: проверяет качество результатов, логирует запросы и расстояния, умеет честно сказать «не нашёл».
- Комбинирует найденные рецепты в единый контекст и строит промпт (system + контекст + вопрос).
- Вызывает GPT, получает ответ и форматирует его для пользователя.

Что у нас есть на выходе части В:

- Рабочая функция RAG (например, rag\_system), которая по любому запросу запускает полный контур retrieval + generation.
- Ответы, опирающиеся на реальную базу рецептов, а не на «фантазии» модели.
- Логи запросов и результатов, которые можно использовать для дальнейшего улучшения системы.

Ключевая идея: онлайн-пайплайн не изменяет базу знаний, а умно использует уже подготовленные эмбединги и индекс, превращая их в понятные ответы для пользователя.

# Итоги: RAG по кулинарным книгам

Что мы построили:

- Полный RAG-конвейер: офлайн-пайплайн (подготовка базы знаний) + онлайн-пайплайн (ответы на вопросы).
- База знаний из кулинарных книг: DocStore + эмбединги + FAISS-индекс + Metadata Store.
- Функцию RAG (`rag_system`), которая по запросу запускает retrieval + generation и выдаёт готовый ответ.

Ключевые принципы, которые важно усвоить:

- Разделение этапов: подготовка данных и индекса офлайн, быстрый ответ онлайн.
- Поиск по смыслу (эмбединги + FAISS), а не только по ключевым словам.
- GPT не «фантазирует», а опирается на переданный контекст из найденных документов.

# ДЗ

Доработать резюме по такой же структуре.

Подключить к чат боту

Бот должен отвечать на вопросы:

«Подбери резюме подходящие на вакансию ....».

«Какие специалисты подходят для выполнения задач ...»

«Найди топ 10 резюме похожих на это ...»

«Какие специалисты обладают компетенциями ...»

«Объясни что делаю такие специалисты простыми словами ...»

Если нет ответа в документах, то должен отвечать правду – нет такой информации.

Подключите RAG по кулинарным книгам к вашему боту.

Подумайте как реализовать классификатор запросов от пользователя, чтобы запросы обрабатывались соответствующим приложением.

[https://disk.yandex.ru/d/\\_kwGDWVMba79Cw](https://disk.yandex.ru/d/_kwGDWVMba79Cw)